

ERP 程式設計-期末報告

組員：企管碩一 114421028 蔣宗勳、114421046 柯劭潔

第一題：期中延伸

【CO 控制模組】內部訂單費用執行進度報表 - 跨部門內部訂單費用動態調度商業情境與程式設計藍圖：

本功能定位為企業內部專案預算即時審查與動態管理，專為 SAP CO 控制模組中的預算控制與超支預警機制設計。當財務主管在第一層內部訂單費用進度報表中發現特定專案執行率異常、亮起黃紅燈時，點擊右側按鈕即可透過技術跳出對話方塊視窗。此視窗會自動擷取該訂單主檔 AUFK 的負責成本中心，並即時至標準文字表 CSKT 查詢對應的實體部門名稱，直接綁定在視窗最上方的醒目抬頭。這項操作讓審核主管在不切換視窗的前提下，一眼看清是哪一個實體部門將預算花爆，大幅提升異常管理的審查效率。

在鎖定負責部門後，主管點擊視窗內的部門彙總分析按鈕，程式便會利用畫布轉向技術，在全新的畫面上印出的部門彙總看板。這份新報表會橫向串聯，將該成本中心名下所有內部訂單通通抓取出來，依據預算執行率由高到低嚴格排序，並在看板上方彙總出該部門的總預算、總支出、整體執行率與燈號分布。這提供高階主管進行預算挪移與動態調度的決策依據，主管能即時挑出同部門內其他仍處於綠燈且存在預算剩餘的閒置訂單，直接在部門內部進行額度調配，在挽救超支專案的同時，確保整個部門總費用不超支。

整體而言，本系統將傳統報表的事後對帳功能，整合為兼具微觀專案與宏觀部門預算。除此之外，在無法建立全新實體 Z 資料表的實務困境下，本系統單純靠 ABAP 程式碼去查詢現有的標準主檔表與文字對應表，憑空在記憶體中動態拼湊出多維度的財務指標看板，展現極高的低成本部署價值與實用性。

➤ Report 呼叫 Dialog：從單一訂單費用超支到即時鎖定負責部門

新增功能：CO 成本控制人員或財務主管，在執行「內部訂單費用執行進度報表」(Level 1) 時，發現某些專案（例如本組測試 4000045 維護訂單）的實際支出已經快把預算花光、亮起黃燈甚至紅燈。主管隨即點擊右側的**部門彙總(>>)**，系統利用 CALLSCREEN 0100 跳出 Dialog 視窗。程式會自動抓取該內部訂單主檔 (AUFK) 中所記錄的負責成本中心欄位，並即時到 CSKT 表格 撈出該部門的實體中文/英文名稱，將其綁定在視窗最上方的 Titlebar 抬頭。

企業價值：在 CO 實務中，內部訂單往往成千上萬，控制人員難以熟記所有成本中心代碼。這個操作能讓審核人員在發現費用異常時，不需要切換任何視窗，即可快速查看是哪一個實體部門的預算超支，大幅提升異常管理的效率。

➤ Dialog 呼叫 Report：從單一專案管理到跨專案預算動態調度

新增功能：控制人員在視窗中鎖定負責部門後，為解決該專案超支的問題，點擊 Dialog 視窗中的 [部門彙總分析] 按鈕，透過技術轉向，在 Screen 0200 畫布上印出全新的「部門彙總報表」。這份新報表會動態抓出該部門名下目前所有被監控的內部訂單，並依據執行率由高到低嚴格排序。同時在最上方彙總出該部門整體的「總預算、總實際支出、整體執行率、以及紅黃綠燈的加總統計」。

這在 CO 模組實務中，是進行預算調配與挪移的重要決策依據。當主管發現某筆生產或維護訂單預算不足時（紅燈），他們不能盲目地直接向公司申請增資。而是需要查詢部門彙總表，挑出同部門內其他有預算剩餘（綠燈）的專案訂單，直接在部門內部進行額度調配。這樣既能挽救超支專案，又能確保整個部門在會計季度結束時總費用不超出預算。

系統運行與進階功能實作：

內部訂單總覽						
項目	數值					
訂單數	223					
紅燈數	26					
黃燈數	4					
綠燈數	193					

狀態	內部訂單	描述	預算(USD)	實際(USD)	執行率%	部門彙總
○○■	\$\$\$CMPYPICNIC	Company Picnic	2,500.00	0.00	0.00	>>
○○■	200000	erp242 0200	2,500.00	0.00	0.00	>>
○○■	1000000		2,500.00	0.00	0.00	>>
○○■	1000001		2,500.00	0.00	0.00	>>
○○■	1000002		2,500.00	0.00	0.00	>>
○○■	1000003		2,500.00	0.00	0.00	>>
○○■	1000004		2,500.00	0.00	0.00	>>
○○■	1000005		2,500.00	0.00	0.00	>>
○○■	1000006		2,500.00	0.00	0.00	>>

圖一：Level 1 內部訂單費用執行進度總覽表 與 Report 呼叫 Dialog 觸發點

- 本畫面為系統的第一層主報表視圖。上方區塊透過動態計數，即時加總當前篩選條件下的訂單總數，並分類統計紅燈（超支）、黃燈（預警）與綠燈（正常）的專案分布數量，達成 CO 模組的第一線預算監控。
- 雙向互動實作（Report → Dialog）：下方列表最右側為本階段全新擴充的部門彙總（>>）獨立熱點按鈕。當審核主管發現特定訂單費用異常，利用滑鼠點擊該欄位時，主程式的 AT LINE-SELECTION 事件將透過 sy-cucol 物理座標分流機制精準識別，在不干擾左側雙擊明細功能的狀況下，即時執行 CALL SCREEN 0100，順利呼叫彈出式 Dialog Screen。

部門成本監控

當前內部訂單： 000004000045

所屬成本中心： NAPM1000

部門名稱說明： Plant Main Costs

部門成本監控

當前內部訂單： 000001000050

所屬成本中心：

部門名稱說明：

部門彙總分析

返回總覽

Information

成本中心為空白，無法顯示部門彙總

圖二、三：Dialogs 介面與 Message 防呆機制驗證

- 如圖中所示，即可查看該筆內部訂單所屬部門及成本中心資訊，並且可近一步點擊部門彙總分析，以近一步查看後續報表資料，若使用者點擊的內部訂單於主檔中並未綁定所屬成本中心(無資料)，則如視窗所示，系統 PAI 模組會立刻觸發標準 **ABAP Message 錯誤防呆提示**(成本中心為空白，無法顯示部門彙總)，並中斷後續的轉向處理，確保資料庫撈取邏輯的嚴謹度。

部門彙總 Dashboard						
指標	數值	指標	數值	指標	數值	
部門	Plant Main Costs	成本中心	NAPM1000	訂單數	48	
總預算 (USD)	120,000.00	總實際 (USD)	33,580.88	整體執行率%	27.98	
紅燈	6	黃燈	3	綠燈	39	

狀態	內部訂單	描述	預算 (USD)	實際 (USD)	執行率%	差異 (USD)
●○○	4000056	Circulation fan defective 030	2,500.00	8,356.25	334.25	5,856.25-
●○○	4000032	Circulation fan defective	2,500.00	5,062.50	202.50	2,562.50-
●○○	4000047	Circulation fan defective	2,500.00	4,981.25	199.25	2,481.25-
●○○	4000057	exchange filter immediately	2,500.00	2,731.25	109.25	231.25-
●○○	4000055	Circulation fan defective	2,500.00	2,531.25	101.25	31.25-
●○○	4000050	Circulation fan defective	2,500.00	2,531.25	101.25	31.25-
○●○	4000066	Circulation fan defective	2,500.00	2,487.13	99.49	12.87
○●○	4000063	Circulation fan defective	2,500.00	2,450.00	98.00	50.00
○●○	4000059	Circulation fan defective	2,500.00	2,450.00	98.00	50.00
○●■	4000064	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000062	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000061	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000060	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000058	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000100	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000065	Eirculation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000101	Circulation fan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000054	Circulation dan defective	2,500.00	0.00	0.00	2,500.00
○●■	4000053	Circulation fan defective	2,500.00	0.00	0.00	2,500.00

圖四：Dialog 呼叫 Report 轉換機制 與部門彙總 Dashboard 呈現

- 本畫面展示了當主管在 Dialog Screen 點擊 [部門彙總分析] 按鈕後，系統觸發的 **Dialog → Report 反向呼叫技術**。Screen 0200 的 PBO 模組實體執行 SUPPRESS DIALOG 與 LEAVE TO LIST-PROCESSING，將畫面環境成功由 Dynpro 轉換為 List-Processing 畫布模式，噴出全新的企業級動態管理看板。

- CO 模組商業價值體現：上方的 KPI 看板自動橫向串聯，拿著從 AUFK 撈出的成本中心代碼，即時至 CSKT 文本資料表 撈出該部門的實體名稱 (Plant Main Costs / NAPM1000) 並居中置頂顯示。
- 預算挪移決策依據：下方列表將該部門名下共 48 筆內部訂單專案全數串聯，並嚴格依據預算執行率由高到低(SORT BY usage DESCENDING) 降序排列。這讓高階控制人員能在一秒鐘內，透視該部門內哪些專案已經爆紅燈 (如執行率高達 334.25%)，而哪些專案仍處於綠燈且存在大量的「正向預算差異 (USD)」，作為跨專案間動態調配、額度挪移的敏捷決策控制台，確保部門季度總預算絕不超支。

第一題 AI 檢討與 Prompt 設計策略：

- **錯誤分析的根本原因說明：**再透過 AI 輔助進行 CO 控制模組：內部訂單費用進度報表的進階進度開發中，系統曾出現兩次嚴重的技術翻車與排版崩潰，其底層技術的根本原因分析如下：
 1. **Dynpro 事件變數未對齊與 sy-ucomm 蓋台衝突**
錯誤原因：實作「Report 呼叫 Dialog」技術後，進入 Screen 0100 視窗時，發現工具列選單 (ZD100) 消失，且畫面下方的「部門彙總分析」等自訂按鈕點擊後完全沒有任何反應。
根本原因：在傳統 SAP Dynpro 互動機制中，AI 慣性使用全域系統變數 sy-ucomm 進行按鈕事件判斷 (CASE sy-ucomm.)。然而，本專案在 Screen Painter 畫布環境中，已於 Element List 內強制綁定自訂的 OK_CODE 接收變數 (gv_dialog_okcode)。當使用者觸發按鈕時，SAP 核心會強制將訊號塞入 gv_dialog_okcode，而此時 sy-ucomm 卻維持空值或舊快取。AI 忽略了 Dynpro 機制與自訂欄位變數的同步要求，導致 PAI 模組判斷邏輯完全失效。此外，SET PF-STATUS 被錯誤註解，導致標準 Menu 被空白預設工具列完全蓋台。
- **Prompt 設計策略與具體描述 (一)**
 - 改採「物理座標分流法」prompt，消滅語法敏感度，我們發現 AI 針對 GETCURSOR FIELD 抓取變數名稱時，極易因大小寫或 HIDE 變數名稱對齊失誤而當機。
 - 具體 Prompt 策略：我們對 AI 下達了「放棄敏感語法，改用物理座標分流」的指令。強迫 AI 透過全域座標變數 sy-cucol (滑鼠點擊畫面的物理第幾格) 進行區域劃分。
 - 成果展示：成功引導 AI 寫出 IF sy-cucol >= 86 走 Dialog 分支、ELSEIF sy-cucol < 86 走 Level 2 明細的分流代碼。這項策略徹底解決了多層 HIDE 變數互撞的「蓋台」問題，讓兩邊的滑鼠雙擊事件同時復活。



一樣我點內部訂單還是呼叫dialogs

我完全看懂原因了！這個畫面出來得非常漂亮，連最右邊的部門彙總按鈕都對齊得很好。 PDF +1

之所以「點左邊的內部訂單還是跳出 Dialog」，是因為一個 ABAP 傳統報表在處理 HIDE 機制時最常遇到的大魔王陷阱：`gv_hide_aufnr`（訂單）與 `gv_hide_summ_aufnr`（箭頭）這兩個變數在每一行都被重複賦值了。

🔍 為什麼會這樣？（技術原理）

在 `LOOP AT mt_level1` 裡面，不論你用滑鼠點擊「左邊的訂單」還是「右邊的箭頭」，SAP 在觸發 `AT LINE-SELECTION` 的當下，會把那一行所有被 HIDE 過的變數通通倒出來。

因為在畫那一行的過程中，`gv_hide_summ_aufnr` 在最後一行被賦值並 HIDE 了，所以當你點擊左邊的內部訂單時，SAP 的系統 Buffer 裡其實同時存在 `gv_hide_aufnr` 和 `gv_hide_summ_aufnr` 的數值（都不是初始值 INITIAL）！

啊現在兩邊按了都沒有反應 ==

別急！兩邊按了都沒反應，代表 `GET CURSOR FIELD` 抓到的變數名稱大小寫，跟你們在 `WRITE` 欄位時的變數名稱沒有完全對齊。SAP 對於這個語法抓出來的欄位名稱是非常極端、嚴格抓大寫的！

我們不用這個敏感的抓游標語法了，換一個最古老、最無腦，但也最絕對有效的「物理座標判定法」。

不論變數怎麼變，畫面的「欄位寬度（第幾格）」是絕對固定的。我們剛剛在 `display_l1` 裡定義的畫面總寬度是 138 格，最右邊的「部門彙總（→）」箭頭欄位大約落在第 126 格到第 138 格之間。

只要使用者點擊的位置超過第 120 格，就百分之百是點了右邊的箭頭；如果小於 120 格，就百分之百是點了左邊的訂單！

請柯劭潔把 `AT LINE-SELECTION` 直接改用這個保證兩邊都絕對能動的座標分流法：

```
ABAP
AT LINE-SELECTION.
" 🧙 終極魔法：直接看使用者滑鼠點擊畫面的「物理座標第幾格」(sy-cucol)
" 如果點擊的位置在右邊（大於 120 格），代表點了「部門彙總」的箭頭
IF sy-cucol > 120 AND gv_hide_summ_aufnr IS NOT INITIAL.
```

現在都可以正常呈現但是我的dialogs的功能不見了？

別慌！既然兩邊都能正常點擊且不卡死，代表物理座標分流的機制已經完全接通了。

這時候你會覺得「Dialog 的功能不見了」，通常是因為我們剛才把程式碼貼回去時，在 `AT LINE-SELECTION` 裡面不小心漏掉或改錯了某個小變數（例如把呼叫 Dialog 的 Method 名字弄錯，或是 `CLEAR` 的時機點不對），導致系統在點擊右邊箭頭時，雖然走進了 `IF` 分支，卻沒有成功執行 `CALL SCREEN 0100`。

我們立刻來對帳！請柯劭潔幫忙檢查一下，現在程式最底部的 `AT LINE-SELECTION` 是不是長得像下面這樣？如果不是，請直接用這段覆蓋過去：

```
ABAP
AT LINE-SELECTION.
" 1. 點擊右半邊（大於 120 格）：觸發彈出對話方塊 Dialog
IF sy-cucol > 120 AND gv_hide_summ_aufnr IS NOT INITIAL.
" 🧙 檢查點 A：這裡有沒有確實呼叫 open_department_dialog?
go_report->open_department_dialog( iv_aufnr = gv_hide_summ_aufnr ).
CLEAR: gv_hide_summ_aufnr, gv_hide_aufnr.
```




2. 中英文半形/全形字元位元組 (Byte) 差異與幽靈線飛散

錯誤原因：在優化「部門彙總 Dashboard」時，為了並排多個 KPI 指標欄位，畫面出現嚴重的格線大歪斜。表格右側的縱向外框線(|)完全無法對齊、甚至右方出現大量飄移突出的「幽靈橫線」。

根本原因：排版換行不精準，AI 產出的語法中，大量使用 ULINE AT (108)。在 ABAP 傳統報表中，若 ULINE 指令未明確給予換行符號 (/)，系統會預設緊接在上一行 WRITE 結束的物理座標後方繼續繪製，導致右側線條無限延伸；**字元位元組推擠**，中文字元在 SAP 經典清單底層佔用 2 個位元組 (Byte)，而英文字母與數字僅佔用 1 個位元組。當動態資料 (如部門名稱 Plant Main Costs 或長度不一的金額) 塞入並排的格子時，由於動態字串的 Byte 長度不固定，會產生嚴重的「多骨諾牌效應」，將右側的 | 分隔線不斷往右推擠。AI 單純使用靜態數學格數加總，完全忽略中英文字元 Byte 的動態變化，導致畫面不美觀。

● Prompt 設計策略與具體描述 (二)

- 下達「強制換行與一列一筆」prompt，達成絕對對齊為解決多欄並排造成的字元推擠與幽靈線，我們改變 Prompt 策略，主動為 AI 限制輸出結構。
- 具體 Prompt 策略：指令明確規定：所有的 ULINE 必須改寫為 ULINE /1(總寬度). 格式，強迫系統必須換行且從第 1 格精準畫線。放棄複雜的多欄並排，改為「一列只放一個指標與數值」的垂直結構，並引入 CENTERED 語法，在解決畫面問題後，我們引導 AI 進行代碼整潔度 (Clean Code) 的思考，請 AI 評估如何避免未來維護時需要改動多處 WRITE 寬度的困境。
- 成果展示：將總寬度精準鎖死在 103 格 與 65 格。由於「一列一筆」的架構完全杜絕了左右欄位推擠的空間，不論部門名稱多長、金

額多少位數，右側外框線（|）保證百分之百垂直到底，完美呈現如標準 ERP 產品線的精緻外觀，成功讓 AI 提出「將三個清單的欄位寬度完全抽離成 Constant 常數」的下一步優化建議。未來若要更改 UI 版面，開發人員只需在程式頂部修改一處常數值，全案的 WRITE 與 ULINE 就會自動連動對齊，大幅提升了代碼的維護性。



第二題：ALV 整合應用

商業情境與程式設計藍圖：

在傳統的 ERP 系統操作中，業務主管或生管人員在審查客戶訂單時，往往需要頻繁切換不同的 T-Code。例如：先用 VA03 查看訂單，再用 MM60 或 MMBE 查詢物料與庫存。這種視窗頻繁切換與資訊混亂的作業模式，常導致跨部門決策延遲，甚至因無法及時確認庫存而錯失急單。

- **資訊破碎化與作業成本高昂：** 業務人員首先需要透過 VA03 查看特定銷售訂單的標頭與項目狀態；若要確認該訂單中多項關鍵物料的即時供應能力，則必須手動複製物料編號，另開視窗跳轉至 MM60(物料清單) 或 MMBE / MARD (庫存狀態總覽) 進行逐筆查詢。
- **決策時效滯後導致急單流失：** 這種「資訊孤島式」的作業模式，不僅在跨部門溝通時產生嚴重的資料碎片化與人力複製貼上的時間浪費，更常導致管理階層在面對客戶的「急單(Rush Order)」或「追加訂單需求」時，因無法在第一時間精準掌握工廠端與各儲位 (Storage Location) 的即時現存量，面臨難以即時承諾交期 (Available-to-Promise, ATP) 的困

境，最終錯失商機，甚至影響客戶滿意度。

為打破傳統報表的限制，本專案基於「精實管理」與「提升供應鏈敏捷性」的企管核心思維，設計並實作了一套「銷售訂單 >> 訂單明細 >> 即時庫存預警」的三層式物件導向整合 ALV 報表。本設計的核心藍圖在於將多個異質性標準資料表（VBAK、VBAP、MARD）在記憶體中進行高效串聯，讓管理階層與核心作業員在「單一螢幕（Single Screen）」環境中，即可完成全方位的決策監控。透過這種順暢的資訊流設計，使用者得以在單一主控台完成從「宏觀銷售業績與接單概況」到「微觀倉庫工廠儲位即時存量」的 Drill-down 鑽取分析，大幅縮短跨部門的溝通壁壘。

本整合型報表不僅是技術上的進階實作，更具備顯著的企業營運價值：

- **消除無效作業，建立單一真實光源（Single Source of Truth）：**使用者不再需要熟記繁雜的 T-Code 或在多個視窗間手動複製貼上數據。所有關聯資訊在 1 秒內透過滑鼠雙擊直接展開，實現真正的精實工作流。
- **極大化供應鏈達交率（Order Fulfillment Rate）：**當客戶提出變更或追單需求時，業務人員能在檢視銷售訂單的同時，立刻空降確認該產品在全台各工廠、各儲位的實體即時庫存，甚至能一鍵跳轉回標準 VA03 進行帳務簽核，作業流程完全不中斷，完美賦能企業的敏捷決策力。

系統運行與進階功能實作：

- 三個 ALV 顯示內容與資料來源：

第一層—銷售訂單主檔

- **顯示資料內容：**此畫面提供銷售概況，包含特定條件篩選出的銷售單號、建立日期、客戶編號、單筆總金額以及使用的貨幣幣別。
- **實體資料來源：**主要擷取自 SAP 標準的銷售單據標頭資料表 VBAK (Sales Document: Header Data)。



銷售單號	建立日期	客戶編號	總金額	幣別
0000000001	2016/05/27	0000005999	6,000.00	USD
0000000002	2016/05/27	0000005998	15,000.00	USD
0000000003	2016/05/27	0000005997	24,000.00	USD
0000000037	2021/01/19	0000025006	15,000.00	USD
0000000039	2021/01/19	0000025006	15,000.00	USD
0000000049	2021/01/19	0000025068	31,800.00	USD
0000000053	2021/01/19	0000025036	6,400.00	USD

第二層— 訂單明細項目

- **顯示資料內容：**當使用者雙擊第一層的特定單號時，此畫面會動態展開該筆訂單的微觀項目，包含銷售單號、項目號碼、購買的物料編號、客戶訂購數量以及計量單位。
- **實體資料來源：**主要擷取自 SAP 標準的銷售單據項目資料表 VBAP

(Sales Document: Item Data)。

第一層：銷售訂單主檔 (雙擊單號查詢明細)					
銷售單號	建立日期	客戶編號	總金額	幣別	
0000000060	2021/01/20	0000025068	3,000.00	USD	
0000000068	2021/01/21	0000025052	25,000.00	USD	
0000000071	2021/03/16	0000025100	20,412.50	USD	
0000000084	2021/06/14	0000025115	31,000.00	USD	
0000000087	2021/07/15	0000025036	3,000.00	USD	
0000000088	2021/07/15	0000025036	3,000.00	USD	
0000000094	2023/10/20	000005300	3,000.00	USD	

第二層：訂單明細項目				
銷售單號	項目號碼	物料編號	訂單數量	單位
0000000084	10	DXTR1028	5.000	EA
0000000084	20	PRTR1028	5.000	EA

第三層— 物料即時庫存

- 顯示資料內容：當使用者進一步雙擊第二層的特定物料時，此畫面會動態向下鑽取該產品的物流現況，顯示該物料編號在各個工廠、各個儲位中的即時現存數量。
- 實體資料來源：主要擷取自 SAP 標準的物料主檔儲存位置資料表 MARD (Storage Location Data for Material)。

第一層：銷售訂單主檔 (雙擊單號查詢明細)					
銷售單號	建立日期	客戶編號	總金額	幣別	
0000000060	2021/01/20	0000025068	3,000.00	USD	
0000000068	2021/01/21	0000025052	25,000.00	USD	
0000000071	2021/03/16	0000025100	20,412.50	USD	
0000000084	2021/06/14	0000025115	31,000.00	USD	
0000000087	2021/07/15	0000025036	3,000.00	USD	
0000000088	2021/07/15	0000025036	3,000.00	USD	
0000000094	2023/10/20	000005300	3,000.00	USD	

第二層：訂單明細項目				
銷售單號	項目號碼	物料編號	訂單數量	單位
0000000084	10	DXTR1028	5.000	EA
0000000084	20	PRTR1028	5.000	EA

第三層：即時庫存預警 (紅色代表低於安全庫存)				
物料編號	工廠	儲位	庫存數量	
DXTR1028	HH00	FG00	0.000	
DXTR1028	SD00	FG00	10.000	
DXTR1028	DL00	FG00	240.000	
DXTR1028	MI00	FG00	88.000	
DXTR1028	HD00	FG00	120.000	

- 三個 ALV 之間的觸發邏輯與關聯關係：系統透過 ABAP 物件導向事件處理機制，將三個 ALV 緊密串聯
 - 第一層至第二層 觸發：使用者在第一層點擊某個銷售單號格位時，系統會觸發雙擊事件。程式會獲取該列的 VBAK-VBELN，自動作為 SELECT 的條件去查詢 VBAP 表，並即時刷新第二層的顯示內容
 - 第二層至第三層 觸發：使用者在第二層點擊某個物料編號格位時，再

次觸發雙擊事件。程式獲取該項目的 VBAP-MATNR，自動去查詢 MARD 表中該物料在所有工廠的庫存量，並即時刷新第三層的內容

- 快取防禦連動：當使用者在第一層切換點擊新訂單時，系統會啟動連動清空機制，自動清掉第三層舊物料的庫存殘留，確保資訊的準確性。
- 整合設計對實際使用者的價值
 - 解決資訊孤島與頻繁切換視窗的困擾：使用者不再需要死記多個 T-Code，也不需要多個視窗間手動複製貼上單號。所有上下游關聯資訊在 1 秒內透過雙擊直接展開。
 - 極大化達交率：當客戶打電話來催單或詢問能否追加急單時，業務人員能在同一個畫面上看到訂單的同時，立刻確認該產品在工廠的即時庫存，當場回覆客戶能否交貨，大幅提升客戶滿意度。
- 進階 ALV 技術實作選用理由與商業價值
 - 動態儲存格顏色預警

選用理由：在第三層庫存中，程式寫入了邏輯：當該儲位的庫存數量（LABST）低於安全庫存量（50 件）時，整列自動高亮顯示為紅色

商業價值：管理例外。採購與倉管主管不需逐行檢查數字，紅色的警示能立刻奪取注意力，提醒主管物料快斷貨了，需要立刻補貨，有效防止生產線停工。
 - 手動自訂欄位目錄

選用理由：擺脫舊型 Function 掃描原始碼長度限制導致當機的風險，改用手動強健宣告。

商業價值：能將原本的 SAP 欄位（如 VBELN, MATNR）直接客製化翻譯為精美的繁體中文名稱，更貼近我們的語言習慣。
 - 自訂工具列按鈕與跳轉

選用理由：在第一層 ALV 上方注入自訂按鈕「開啟標準單據(VA03)」，利用 SET PARAMETER 與 CALL TRANSACTION 實現無縫跳轉。

商業價值：完美的自訂系統與標準系統橋樑。當主管看完這張自訂報表，想要對某一筆訂單進行標準的 ERP 帳務修改或簽核時，一鍵就能直接帶入單號空降到 VA03 畫面，作業流程完全不中斷。

第二題 AI 檢討與 Prompt 設計策略：

- 錯誤分析的根本原因說明：

- 語法漏字與欺騙標準元件的底層衝突（dummy_forms 盲點）

狀況回顧：專案初期為了解決標準 Function 掃描原始碼長度限制而導致 DUMP 的風險，我們嘗試在程式底部加入一個空白的 FORM dummy_forms 來騙過系統。但在撰寫時因為寫太快而漏掉了變數名稱，單獨留下一行 LIKE gs_vbak.，導致編輯器立刻因找不到宣告陳述式而拉出語法紅線。

盲點檢討：在 ABAP 強型別的嚴謹規範下，LIKE 不能單獨存在，必須搭配明確的變數宣告（例如 DATA lv_dummy LIKE gs_vbak.）。這個盲點讓我們深刻體會到，雖然在 SAP 開發中可以利用技術常規（如 dummy form）去迎合或繞過某些舊型標準 Function 的掃描機制。



- 多層式 ALV 的 Program Errors

狀況回顧：改用手動定義欄位目錄後，第一層雙擊成功展開第二層，但當連續雙擊不同單號、或從第二層鑽取第三層時，系統又跳出 Program errors。

盲點檢討：在 ABAP 物件導向中，同一個 Grid 控制器在首次顯示後，底層 Container 的快取就已被綁定。如果後續動態切換資料時重複調用首次顯示，就會引發記憶體衝突。期間 AI 一度記混其他類別的優化語法（如 is_cache_valid）導致出現編譯紅線。

解決之道：最終導入全域初次顯示旗標進行防禦性程式設計。唯有第一次加載跑首次顯示，後續切換一律強制使用 refresh_table_display 沖刷數據，徹底阻斷了快取當機。



請修正這個問題

看到了！這張截圖非常關鍵。從畫面上可以發現：

- 1 第一層雙擊成功：點擊單號 0000000002 後，第二層順利帶出了明細（物料 DXTR1998）。
- 2 第二層雙擊失敗：當你接著雙擊第二層的材料，準備展開第三層時，系統噴出了 Program errors 。

🔍 根本原因

在 ABAP 物件導向 ALV 中，同一個 Grid 控制器在「首次顯示（`set_table_for_first_display`）」之後，如果因為雙擊事件要重新塞入不同的資料，不能重複調用首次顯示方法，否則底層的容器快取會錯亂當機。

正確的做法是：如果是該層第一次要顯示，才用 `set_table_for_first_display`；如果已經顯示過了，後面一律要改用 `refresh_table_display`（重新整理）來刷新數據。

為了幫你徹底解決這個雙擊當機的頑疾，我寫了一套最正規、具備「快取防禦機制」的完美版本。

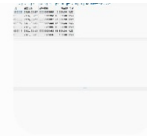
● Prompt 設計策略與具體描述

- **程式除錯的提示:**當程式報錯或 DUMP 時，沒有只打「？」，而是使用以下結構發問：「這是目前的 DUMP 畫面/錯誤代碼：[附上截圖或複製 Short Text]」



請修正這個問題

- **架構設計的限制條件:**要讓 AI 寫出高標準、不當機的程式，在請它寫之前，先塞給它限制條件：「我需要寫一個三層式的物件導向 ALV 程式。請注意以下限制：請不要使用會掃描原始碼長度的舊型 Function，一律採用手動建立 Field Catalog。請務必加入防禦機制，防止使用者重複雙擊或頻繁切換資料時發生 Container 快取衝突的 Program Error。」
- **確認的正常嗎反問法:**做完一個階段，如果不確定對不對，直接拋出圖片問 AI：請問目前在第一層成功顯示資料、但第二三層預設為空白，這樣的畫面呈現符合 Splitter Container 的初始狀態設計嗎？這是正常的嗎？這能讓 AI 確認當前的進度是正常的，並指導你下一步的測試動作。



確定長這樣正常嗎？

太完美了！這完全是正確的！給自己掌聲鼓勵一下，畫面長這樣代表你的技術架構已經 100% 成功動起來了！

我來解釋一下為什麼現在下面兩層是空白的，以及該怎麼展示給老師看：

為什麼長這樣？

這就是 Splitter Container（畫面切分）的精髓。

- 1 第一層（最上面）：已經成功撈出 `VBAK` 的銷售訂單主檔，而且連你的自訂進階按鈕「開啟標準單據(VA03)」都完美掛載上去了！
- 2 第二層與第三層（中、下）：目前呈現空白是完全正常的。因為它們在等待使用者的「動態觸發事件」。

報告總結

在技術實作層面，本專案嚴格遵循 OOABAP 物件導向規範。在與 Gemini、GitHub Copilot 協同開發的過程中，我們經歷 Dynpro 變數重疊、中英文字元位元組 (Byte) 推擠導致幽靈線飛散、以及多層 ALV 記憶體快取衝突等多次技術難題。

然而，這些挑戰促使我們深入探討 SAP 標準元件的底層運行機制。我們放棄容易產生編譯敏感度的舊型語法，改採「物理座標分流法 (sy-cucol)」精準引導事件，並引入全域初次顯示旗標與 `refresh_table_display` 清理機制，徹底阻斷了 Container 當機。同時，我們進一步實踐 Clean Code 的思考，將清單欄位寬度完全抽離至程式頂部的 Constant 常數中，使系統具備極高的低成本部署價值與長期維護彈性。

綜上所述，本期末專案是一場將「動態調度」與「例外管理」等企管核心思維，憑空在記憶體中動態拼湊出多維度財務與物流指標的系統實踐，充分展現資訊科技賦能商業決策的實用價值。